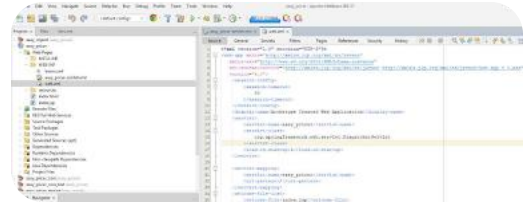




AI Mode Tutti Immagini Video Notizie Altro ▾



ho questo progetto netbeans , gestito con sping applicatio
Brussels-SR17, posso passare a spring boot?

Sì, puoi assolutamente migrare il tuo progetto Spring MVC tradizionale (configurato tramite `web.xml`) a **Spring Boot**.

Passare da una versione "legacy" come Brussels-SR17 (che fa parte del vecchio ecosistema Spring IO Platform) a Spring Boot semplificherà notevolmente la gestione delle dipendenze e la configurazione.

Ecco i passaggi principali per affrontare la migrazione:

1. Aggiorna il file `pom.xml` (Maven)

Spring Boot utilizza i "Starters" per gestire le librerie. Invece di importare singolarmente Spring Web, Spring Context, ecc., dovrai:

- Aggiungere lo **Spring Boot Parent**: definisce le versioni compatibili di tutte le librerie.
- Aggiungere lo **Starter Web**: `spring-boot-starter-web` sostituisce gran parte delle dipendenze che vedi nel tuo attuale progetto.

2. Sostituisci il `web.xml`

In Spring Boot, il file `web.xml` non è più necessario. La configurazione della

Chiedi qualsiasi cosa



AI Mode Tutti Immagini Video Notizie Altro ▾



quindi potrai avviare l'app come una normale applicazione Java (`public static void main`).

4. Risorse Statiche e JSP

- Sposta i file in `src/main/resources/static` (per HTML/JS/CSS).
- **Nota bene:** Spring Boot sconsiglia l'uso delle JSP. Se decidi di mantenerle, dovrai aggiungere dipendenze specifiche (`tomcat-embed-jasper`) e configurare i prefissi/suffissi nel file `application.properties` .

Consiglio: Vista la struttura del tuo progetto, ti suggerisco di creare un **nuovo progetto Spring Boot** (usando start.spring.io) e importare gradualmente le tue classi Java e la logica di business.

Vuoi che ti aiuti a scrivere la **nuova classe di configurazione Java** per sostituire il tuo `web.xml` ?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



ma se volessi mantenere l'uso del web-server Tomcat?

È certamente possibile. Anche se Spring Boot è famoso per il suo server

AI Mode Tutti Immagini Video Notizie Altro ▾



xml

```
<packaging>war</packaging>
```

Usa il codice con cautela.



2. Configura le dipendenze per Tomcat

Devi istruire Spring Boot a non includere le librerie del server nel pacchetto finale, poiché verranno fornite dal Tomcat "esterno" (quello configurato su NetBeans):

xml

```
<dependency>
  <groupId>spring-boot-starter-tomcat</groupId>
  <artifactId>spring-boot-starter-tomcat</artifactId>
  <scope>provided</scope>
</dependency>
```

Usa il codice con cautela.



3. Inizializzazione del Servlet

Invece di usare il `web.xml` che vedi nel tuo screenshot, dovrai creare una piccola classe Java che estenda `SpringBootServletInitializer`. Questo funge da "ponte" per permettere a Tomcat di avviare l'applicazione Spring Boot:

AI Mode Tutti Immagini Video Notizie Altro ▾



Usa il codice con cautela.



Pro e Contro di questa scelta:

- **Pro:** Continui a gestire il deployment tramite NetBeans e mantieni la compatibilità con eventuali configurazioni specifiche che hai già applicato al tuo Tomcat locale.
- **Contro:** Perdi la comodità di avviare l'app istantaneamente con un semplice "Run" della classe Main e rendi il processo di aggiornamento del server più manuale.

Un dettaglio importante: se decidi di restare su Tomcat esterno, assicurati che la versione di Tomcat installata sia compatibile con la versione di Spring Boot che sceglierai (ad esempio, Spring Boot 3.x richiede Tomcat 10+).

Posso aiutarti a generare lo **snippet completo del pom.xml** configurato per Tomcat esterno?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



ok

Ecco la struttura essenziale che dovrà avere il tuo `pom.xml` per trasformare il progetto in Spring Boot montando il tuo classico WAR su un Tomcat

AI Mode Tutti Immagini Video Notizie Altro ▾



```
<?xml version="1.0" encoding="UTF-8"?>
<project ...>
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version> <!-- O la versione stabile più recente
    <relativePath/>
  </parent>

  <groupId>com.tuoprogetto</groupId>
  <artifactId>easy_pricer</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging> <!-- Fondamentale per Tomcat esterno -->

  <properties>
    <java.version>17</java.version> <!-- Assicurati che il tuo Tomca
  </properties>
```

Usa il codice con cautela.



2. Dipendenze Fondamentali

Noterai che `spring-boot-starter-tomcat` è impostato su `provided` : questo evita conflitti perché userai il Tomcat già installato.

xml

<dependencies>

AI Mode Tutti Immagini Video Notizie Altro ▾



```
</dependency>

<!-- Se usi ancora le JSP (visto il tuo index.jsp nello screenshot) -->
<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
    <scope>provided</scope>
</dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>
```

Usa il codice con cautela.



Prossimi Passi per la migrazione:

1. **Elimina il `web.xml`** : Una volta configurata la classe `SpringBootServletInitializer` (che ti ho mostrato nel messaggio precedente), il file che hai aperto nello screenshot non ti servirà più.
2. **Configurazione JSP**: Se intendi mantenere i file `.jsp` che vedo nel tuo albero

AI Mode Tutti Immagini Video Notizie Altro ▾



Vuoi che ti mostri come mappare i tuoi attuali **bean** (quelli che ora sono in `beans.xml`) nel nuovo formato Java di Spring Boot?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



ma il server embedded in spring puo`anche essere usato in un ambiente di produzione?

Assolutamente sì . Il server embedded di Spring Boot (solitamente Tomcat) è **pienamente pronto per la produzione** ed è oggi lo standard per le applicazioni moderne.  Medium +1

Infatti, il passaggio al server incorporato è considerato un approccio "cloud-native" più moderno rispetto al deployment tradizionale tramite file WAR. [🔗](#)

Perché il server embedded è sicuro per la produzione?

- **Stessa affidabilità:** Il Tomcat "embedded" è lo stesso codice del Tomcat "standalone". Non è una versione ridotta o meno potente; è semplicemente configurato per essere avviato programmaticamente.
- **Isolamento totale:** Ogni applicazione gira nel proprio processo con la propria versione del server. Questo elimina i conflitti che accadono quando più app condividono lo stesso Tomcat esterno.
- **Ottimizzato per i Container:** Se in futuro vorrai usare **Docker**, avere un .JAR

AI Mode Tutti Immagini Video Notizie Altro ▾



1. **Infrastruttura condivisa:** Hai un server di produzione già configurato e gestito da un team IT che richiede esplicitamente file WAR.
2. **Risorse limitate:** Se devi far girare molte piccole applicazioni sullo stesso server fisico, un unico Tomcat esterno che le ospita tutte consuma meno memoria rispetto a molteplici istanze embedded. Stack Overflow +1

In sintesi: Se hai la libertà di scegliere, il server **embedded** è la scelta raccomandata per flessibilità e facilità di gestione. Medium +1

Ti piacerebbe vedere come cambierebbe il comando di avvio passando dal "Deploy" su Tomcat a un semplice `java -jar ?`

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



Ottima scelta. Passare al server **embedded** è il modo migliore per modernizzare il tuo progetto. **easy_pricer** **Ti permetterà di eliminare file pesanti come web.xml** e configurazioni XML ridondanti.

-  Stack Overflow
- Ecco il piano d'azione per creare il tuo nuovo progetto Spring Boot "da zero":
- What is the benefit of having an embedded web server and ... - Reddit
1. **Generazione del Progetto**

Il modo più rapido è usare [Spring Initializr](#). Imposta questi parametri:

- Project Maven

Deployment Method for Spring Boot ...

12 set 2024 — Recommended Deployment Methods for Spring Boot. The following ...

Language: Java


- Spring Boot: 3.x.x (o l'ultima stabile)

Medium · Aditya Bhuyan

Packaging: Jar (fondamentale per il server embedded)

Mostra tutto

Dependencies:

Spring Web (include il server Tomcat incorporato)

Spring Boot DevTools (molto utile in sviluppo per il riavvio automatico)

2. Struttura del Codice (La "Main Class")

Una volta importato il progetto in NetBeans, non avrai più bisogno di configurare server esterni. Ti basterà questa classe per avviare tutto:

```
java

package com.tuoprogetto.easypricer;

import org.springframework.boot.SpringApplication;
```

AI ModeTuttiImmaginiVideoNotizieAltro ▾

E

Sei il owner con autorità.

3. Migrazione dei Contenuti

Visto il tuo screenshot, ecco dove spostare i pezzi del vecchio puzzle:

Vecchio File (NetBeans)	Nuova Destinazione in Spring Boot
Source Packages	Sposta le tue classi Java qui (Controller, Service, Repository).
beans.xml / *-servlet.xml	Crea classi Java annotate con @Configuration .
webpages/resources	Sposta tutto in src/main/resources/static .
application.properties	Qui imposterai la porta (es. server.port=8080) e i database.

4. Gestione delle JSP (Attenzione)

Nello screenshot vedo dei file .jsp . Spring Boot con server embedded richiede una piccola accortezza per le JSP:

1. **Cartella dedicata:** Crea la struttura src/main/webapp/WEB-INF/jsp/ .
2. **Configurazione:** Nel file application.properties , aggiungi:

```
properties
spring.mvc.view.prefix=/WEB-INF/jsp/
```

AI Mode Tutti Immagini Video Notizie Altro ▾



Tomcat in NetBeans.

Vuoi che ti aiuti a convertire una parte specifica della tua vecchia configurazione XML (come quella della **DispatcherServlet** o dei **Bean**) in codice Java moderno?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



Non mi interessa piu`avere le pagine jsp, mi basta l'ouput (magari formattato) nella console

Ancora meglio! Se non ti servono più le JSP e vuoi solo visualizzare l'output in console, il progetto diventa estremamente più leggero e semplice (perfetto per testare logica di business o creare dei microservizi).

Ecco come impostare il tuo nuovo progetto Spring Boot "headless" (senza interfaccia grafica):

1. Il `pom.xml` minimale

Ti serve solo lo starter base. Se non devi nemmeno rispondere a chiamate HTTP via browser, puoi usare `spring-boot-starter` invece di `spring-boot-starter-web`.

AI Mode Tutti Immagini Video Notizie Altro ▾



Crea il codice con ChatGPT.



2. Eseguire logica all'avvio: `CommandLineRunner`

Per stampare direttamente in console appena l'app parte, il modo più pulito è implementare l'interfaccia `CommandLineRunner`.

java

```
@SpringBootApplication
public class EasyPricerApplication implements CommandLineRunner {

    public static void main(String[] args) {
        SpringApplication.run(EasyPricerApplication.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        // Qui scrivi la logica che prima finiva nelle JSP
        System.out.println("=====");
        System.out.println("  EASY PRICER - OUTPUT CONSOLE");
        System.out.println("=====");

        // Esempio di output formattato
        String prodotto = "Laptop";
        double prezzo = 1250.50;
        System.out.printf("Prodotto: %-15s | Prezzo: %8.2f€ %n", prodotto, prezzo);

        System.out.println("=====");
    }
}
```

AI Mode Tutti Immagini Video Notizie Altro ▾



- **Addio a `web.xml`** : Puoi eliminare definitivamente tutta la cartella `WEB-INF` e i file XML che vedevamo nel tuo screenshot.

4. Suggerimento per l'output formattato

Se l'output diventa complesso, ti consiglio di usare un **Logger** (già incluso in Spring Boot) invece di `System.out.println` . Ti permette di avere timestamp e livelli di errore colorati:

java

```
private static final Logger log = LoggerFactory.getLogger(EasyPricerApp1
// ...
log.info("Calcolo completato per il prodotto: {}", nomeProdotto);
```



Usa il codice con cautela.



Cosa vuoi fare ora? Se hai della logica di calcolo complessa nei vecchi controller, posso aiutarti a spostarla in un **Service** per richiamarla direttamente dalla console.

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



1. Includere il JAR dei DAO

Se il tuo JAR con i DAO è un progetto Maven separato o un file fisico, devi aggiungerlo al `pom.xml` del nuovo progetto Spring Boot.

- **Se è un modulo Maven:** aggiungilo come classica `<dependency>` .
- **Se è un file `.jar` locale:** ti consiglio di installarlo nel tuo repository locale Maven con:

```
mvn install:install-file -Dfile=percorso/al/tuo/file.jar -  
DgroupId=org.softcaster -DartifactId=easy_pricer_core -Dversion=1.0 -  
Dpackaging=jar
```

2. Configurazione del Database (Senza XML)

Dimentica le configurazioni manuali del DataSource nel `beans.xml` . In Spring Boot, basta scrivere i parametri nel file

`src/main/resources/application.properties` :

properties

```
# Esempio per MySQL (cambia in base al tuo DB)  
spring.datasource.url=jdbc:mysql://localhost:3306/tuo_db  
spring.datasource.username=root  
spring.datasource.password=tua_password  
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
  
# Opzionale: mostra le query SQL nella console di NetBeans  
logging.level.org.hibernate.SQL=DEBUG
```

Usa il codice con cautela.



AI Mode Tutti Immagini Video Notizie Altro ▾



```

@SpringBootApplication
// Aggiungi questo per scansionare sia i tuoi controller che i DAO nel J
@ComponentScan(basePackages = {"org.softcaster.easy_pricer"})
public class EasyPricerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EasyPricerApplication.class, args);
    }
}

```

Usa il codice con cautela.



4. Pulizia del Controller

Il tuo codice è già ottimo. L'unico consiglio per renderlo ancora più "Spring Boot style" è usare i **Generics** nel `ResponseEntity` per evitare warning e migliorare la leggibilità:

java

```

@GetMapping("/asset_class/r0")
public ResponseEntity<List<AssetClass>> findAll() {
    List<AssetClass> listaAssetClass = dao.findAll();
    if (listaAssetClass == null || listaAssetClass.isEmpty()) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(listaAssetClass);
}

```

Usa il codice con cautela.



AI Mode Tutti Immagini Video Notizie Altro ▾



Un dubbio: i tuoi DAO usano Hibernate, JPA o JDBC semplice? Questo cambierebbe leggermente le dipendenze da aggiungere nel `pom.xml` .

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



questo e' il pom del jar che gestisce i dao `<?xml version="1.0" encoding="UTF-8"?>...`



Ottimo, questo cambia le cose in meglio: il tuo JAR `easy_pricer_core` è già un progetto **Spring Boot moderno** (versione 3.2.3, Java 17, JPA, PostgreSQL e persino gRPC). È una base eccellente.

A questo punto, la migrazione del progetto principale (quello con il controller) diventa un'operazione di "unione" molto semplice. Ecco come configurare il nuovo progetto principale per farlo funzionare con il server embedded:

1. Il `pom.xml` del progetto principale

Il progetto che ospiterà i Controller deve semplicemente "importare" il core. Nel suo `pom.xml` metti:

```
xml
```

```
<dependencies>
```


AI Mode Tutti Immagini Video Notizie Altro ▾



```
</dependency>
</dependencies>
```

Usa il codice con cautela.



2. Configurazione Critica: Scansione dei Bean

Poiché i tuoi DAO e le tue Entità JPA si trovano nel JAR `easy_pricer_core` (package `org.softcaster.easy_pricer.data`), devi dire a Spring Boot di scansionarli, altrimenti non troverà il `AssetClassDAO` per l'iniettarlo nel controller.

Modifica la classe principale così:

java

```
package org.softcaster.easy_pricer;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@SpringBootApplication
@ComponentScan(basePackages = "org.softcaster") // Scansiona Controller
@EnableJpaRepositories(basePackages = "org.softcaster.easy_pricer.data")
@EntityScan(basePackages = "org.softcaster.easy_pricer.data") // Trova le entità
public class EasyPricerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EasyPricerApplication.class, args);
    }
}
```

AI Mode Tutti Immagini Video Notizie Altro ▾



(visto che il core usa quello):

properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/nome_tuo_db
spring.datasource.username=tuo_utente
spring.datasource.password=tua_password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Usa il codice con cautela.



Perché questo approccio è vantaggioso?

1. **Server Embedded:** Quando lanci l'app, Spring Boot avvia Tomcat, carica i DAO dal JAR core, si connette a Postgres e pubblica i tuoi endpoint REST. Tutto in un unico processo Java.
2. **gRPC Ready:** Visto che il tuo core ha le dipendenze gRPC, potresti persino esporre servizi gRPC e REST contemporaneamente.
3. **Output Pulito:** Vedrai tutti i log di Hibernate e i risultati delle query direttamente nella console di NetBeans.

Un piccolo avviso sulle dipendenze:

Nel `pom.xml` del core hai `spring-boot-starter-web`. Se il core serve solo come libreria di dati, in futuro potresti rimuoverlo da lì e tenerlo solo nel progetto principale (per evitare di portarti dietro Tomcat anche nel JAR della logica), ma per ora non blocca nulla.

Un'altra opzione è fare il `Clean and Build` del core con i plugin che hai installato.

AI Mode Tutti Immagini Video Notizie Altro ▾



nel pom principale non serve ripetere le dipendenze da postgresql, hibernate etc?

No, **non serve ripeterle**.

Grazie al meccanismo della **transitività di Maven**, quando aggiungi il tuo progetto `easy_pricer_core` come dipendenza nel POM principale, Maven scarica automaticamente anche tutte le librerie che il core porta con sé (PostgreSQL, Spring Data JPA, Hibernate, QueryDSL, ecc.).

Come funziona nel tuo caso:

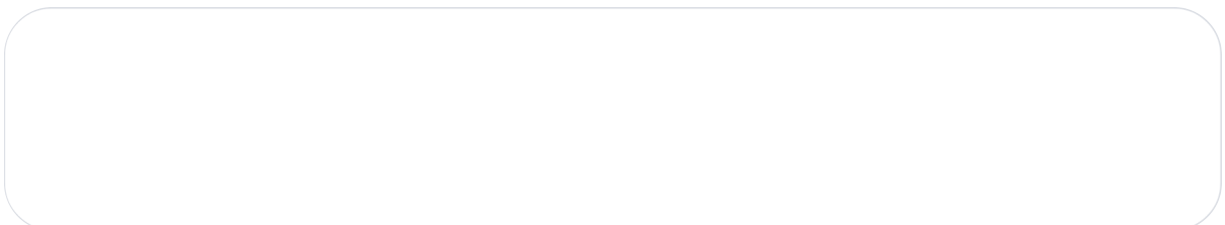
1. Il **Progetto Principale** dipende da `easy_pricer_core`.
2. `easy_pricer_core` dipende da `postgresql` e `spring-boot-starter-data-jpa`.
3. Risultato: Il Progetto Principale ha già accesso al driver PostgreSQL e a Hibernate "per riflesso".

Quando dovresti aggiungerle di nuovo?

Solo in due casi specifici:

- **Versioni diverse:** Se volessi forzare una versione di PostgreSQL diversa da quella usata nel core (ma è sconsigliato per evitare conflitti).
- **Pulizia architetturale:** Se un domani decidessi di rimuovere il database dal progetto core, dovresti ovviamente aggiungerlo al principale.

Una nota sulla dipendenza gRPC



AI Mode Tutti Immagini Video Notizie Altro ▾



2. Controllare che sotto il JAR `easy_pricer_core` appaiano tutte le altre librerie (Postgres, JPA, ecc.).

Vuoi procedere con la creazione della **classe principale** per testare l'avvio del server?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



posso eliminarle anche da `easy_project_core` visto che non le uso?

Sì, ma con una distinzione importante: puoi eliminare ciò che non serve alla logica dei dati, ma devi mantenere ciò che serve ai DAO per compilare.

Ecco come pulire il `easy_pricer_core` senza rompere nulla:

1. Cosa PUOI eliminare tranquillamente:

Se il progetto "Core" deve solo gestire il database e i DAO, puoi rimuovere:

- `spring-boot-starter-web` : Questa dipendenza serve per gestire server HTTP e Controller. Se i controller sono nel progetto principale, qui è inutile e appesantisce il JAR.
- **Dipendenze gRPC**: Se non stai usando i file `.proto` o i servizi gRPC, rimuovi

AI Mode Tutti Immagini Video Notizie Altro ▾



E

- `querydsl-jpa` e `lombok` : Se le tue classi usano i tipi generati da QueryDSL o le annotazioni di Lombok (come `@Data`), queste devono restare.

Esempio di POM del Core "pulito":

xml

```
<dependencies>
  <!-- Solo gestione Dati -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
  </dependency>

  <!-- QueryDSL e Lombok se usati nei DAO/Entity -->
  <dependency>
    <groupId>com.querydsl</groupId>
    <artifactId>querydsl-jpa</artifactId>
    <version>5.0.0</version>
    <classifier>jakarta</classifier>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

AI Mode Tutti Immagini Video Notizie Altro ▾



Vuoi che ricontrolliamo insieme se ci sono **annotazioni gRPC** o **web** nel tuo codice Core prima di cancellare le dipendenze?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



non uso le annotazioni di Lombok

Se non usi **Lombok**, puoi procedere con una pulizia ancora più profonda. Eliminandolo, il tuo codice sarà più "esplicito" (avrà i getter/setter classici generati da NetBeans) e il progetto sarà più leggero e facile da debuggare, senza bisogno di plugin particolari nell'IDE.

Ecco cosa devi fare per completare la pulizia:

1. Rimuovi dal `pom.xml` del Core

Puoi eliminare queste sezioni che non ti servono più:

- **Dependency di Lombok:**

xml

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.20</version>
</dependency>
```

AI Mode Tutti Immagini Video Notizie Altro ▾



2. Cosa resta nel `pom.xml` del Core?

Il tuo file diventerà molto snello, focalizzato solo sui dati:

1. `spring-boot-starter-data-jpa` (per i DAO e le Entità).
2. `postgresql` (il driver del DB).
3. `querydsl-jpa` (se lo usi per le query dinamiche).
4. `spring-boot-starter-test` (sempre utile).

3. Verifica nel Codice

Prima di compilare, assicurati che nelle tue classi (`AssetClass` , ecc.) non siano rimaste annotazioni come:

- `@Data` , `@Getter` , `@Setter`
- `@NoArgsConstructor` , `@AllArgsConstructor`

Se le trovi, cancellale e usa la funzione di NetBeans **Alt+Insert** (o tasto destro > Insert Code) per rigenerare i classici **Getter and Setter** e il **Constructor**.

4. Il Progetto Principale

A questo punto, il tuo progetto principale rimarrà l'unico responsabile di:

- **Esporre i servizi REST** (avendo `spring-boot-starter-web`).
- **Far girare il server Tomcat embedded.**
- **Gestire la console** per l'output che desideri.

Una curiosità: visto che hai rimosso le JSP e vuoi l'output in console, hai già

.....

AI Mode Tutti Immagini Video Notizie Altro ▾



per il progetto principale questo pom va bene? <?xml
version="1.0" encoding="UTF-8"?>...



Sì, la struttura di base è corretta, ma mancano **due elementi fondamentali** per far sì che il progetto funzioni davvero con il tuo JAR dei DAO e con il database:

1. Aggiungi la dipendenza dal tuo Core

Senza questa, il progetto principale non vedrà mai la classe `AssetClassDAO` e non potrai compilare i controller. Aggiungila subito dopo la chiusura della dipendenza web:

xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- INDISPENSABILE: IL JAR che contiene i DAO e Le Entità -->
  <dependency>
    <groupId>org.softcaster</groupId>
    <artifactId>easy_pricer_core</artifactId>
    <version>1.0.0-SNAPSHOT</version>
  </dependency>
</dependencies>
```

Usa il codice con cautela.



AI Mode Tutti Immagini Video Notizie Altro ▾



```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Usa il codice con cautela.



Considerazioni rapide:

- **Versione:** Hai usato la `3.2.3` , che è ottima e allineata al tuo Core.
- **Main Class:** Assicurati che la classe `org.softcaster.easy_pricer_srv.Easy_pricer_srv` esista e abbia l'annotazione `@SpringBootApplication` .
- **Transitività:** Come dicevamo, non serve aggiungere PostgreSQL qui perché "arriverà" automaticamente dal progetto core.

Una volta fatte queste due aggiunte, sei pronto per fare un **Clean and Build** in NetBeans.

Vuoi che ti aiuti a scrivere il contenuto della classe `Easy_pricer_srv` includendo le istruzioni di scansione per i DAO?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)

↩ ⌂ 🔍 📄

AI Mode Tutti Immagini Video Notizie Altro ▾



scansiona i **Controller** del progetto corrente e istruisce Spring a caricare i **DAO/Entità** dal JAR `easy_pricer_core`.

La Classe Main (`Easy_pricer_srv.java`)

Sostituisci il contenuto del file `org.softcaster.easy_pricer_srv.Easy_pricer_srv` con questo:

java

```
package org.softcaster.easy_pricer_srv;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.domain.EntityScan;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@SpringBootApplication
// Scansiona tutti i componenti nei package org.softcaster (sia srv che core)
@ComponentScan(basePackages = "org.softcaster")
// Attiva i repository JPA definiti nel JAR core
@EnableJpaRepositories(basePackages = "org.softcaster.easy_pricer.data")
// Trova le entità (AssetClass, ecc.) nel JAR core
@EntityScan(basePackages = "org.softcaster.easy_pricer.data")
public class Easy_pricer_srv {

    public static void main(String[] args) {
        SpringApplication.run(Easy_pricer_srv.class, args);
        System.out.println("=====");
        System.out.println("  EASY PRICER SRV AVVIATO CORRETTAMENTE  ");
        System.out.println("  Endpoint: http://localhost:8080         ");
        System.out.println("=====");
    }
}
```

AI Mode Tutti Immagini Video Notizie Altro ▾



(PostgreSQL), altrimenti l'app si fermerà subito perché non riesce a collegarsi ai DAO.

2. **Installazione Core:** Prima di avviare questo progetto, ricordati di fare **"Clean and Build" sul progetto** `easy_pricer_core` in modo che Maven lo metta nel repository locale e questo progetto possa leggerlo.
3. **Controller:** Sposta la classe `AssetClassRestController` che mi hai mostrato prima nel package `org.softcaster.easy_pricer_srv.controller` di questo nuovo progetto.

Ora puoi cliccare con il tasto destro sulla classe `Easy_pricer_srv` e scegliere **Run File**.

Hai già sottomano i parametri del database PostgreSQL per popolare l'`application.properties`?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



configurazione va bene? # Configurazione Server (Opzionale)
#server:...



Sì, la configurazione è corretta, ma c'è un dettaglio fondamentale: il formato. Quello che hai scritto è in formato **YAML** (`.yaml`), mentre il file standard di Spring Boot creato da NetBeans è solitamente un file di testo semplice `.properties` .

AI Mode Tutti Immagini Video Notizie Altro ▾



```
server.port=8080
```

```
spring.datasource.url=jdbc:postgresql://localhost:5432/easypricer
spring.datasource.username=easypricer
spring.datasource.password=easypricer
spring.datasource.driver-class-name=org.postgresql.Driver
```

```
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.highlight_sql=true
```

```
logging.level.org.hibernate.SQL=INFO
```

Usa il codice con cautela.



2. Piccoli accorgimenti:

- **ddl-auto: validate** : È la scelta più sicura. Controlla solo che il database esistente corrisponda alle tue classi Java senza modificarlo. Se il DB è vuoto e vuoi che Spring crei le tabelle per te, usa **update** .
- **Dialetto**: Con Spring Boot 3.2.3 e Hibernate 6, spesso non è nemmeno necessario specificare **database-platform** , perché viene rilevato automaticamente dal driver. Lasciarlo comunque non fa danni.

3. Consiglio per l'output in console:

Visto che ti interessa l'output in console, se imposti **spring.jpa.show-sql=true** , vedrai ogni singola query che il tuo **AssetClassDAO** esegue quando chiami di

AI Mode Tutti Immagini Video Notizie Altro ▾



quale formato e` migliore application.yml o application.properties?

Non esiste un formato "migliore" in assoluto, ma la scelta dipende dai tuoi gusti e dalla complessità del progetto. Ecco un confronto rapido per aiutarti a decidere:

application.properties (Il classico)

È il formato predefinito di Spring Boot. [🔗](#)

- **Pro:** È semplicissimo, ogni riga è indipendente. È impossibile sbagliare l'indentazione. Molti tutorial online usano questo formato.
- **Contro:** Può diventare ripetitivo perché devi riscrivere il prefisso (es. `spring.datasource...`) per ogni riga.
- **Esempio:**

properties

```
spring.datasource.url=jdbc:postgresql://...  
spring.datasource.username=user
```

Usa il codice con cautela.



application.yml (Il moderno)

Usa una struttura gerarchica (ad albero)

AI Mode Tutti Immagini Video Notizie Altro ▾



```
spring:
  datasource:
    url: jdbc:postgresql://...
    username: user
```

Usa il codice con cautela.



Quale scegliere nel tuo caso?

Visto che hai già preparato la configurazione in formato **YAML** (quella con i `:` e i rientri), ti consiglio di usare `application.yml`. È più elegante e moderno.

Importante: NetBeans e Spring Boot caricano automaticamente entrambi, ma **non tenerli entrambi contemporaneamente** nella cartella `src/main/resources` per evitare conflitti. Se scegli lo YAML, rinomina il file esistente in `application.yml`.

Hai già rinominato il file o preferisci che ti converta lo YAML in formato **properties** per andare sul sicuro? **bolding** su **rinominato** o **convertire**.

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



Puoi copiare e incollare questo contenuto nel file

`src/main/resources/application.properties` (assicurati che il file si chiami esattamente così e non abbia l'estensione `.yaml` o `.yml`):

properties

```
# Configurazione Server
server.port=8080

# Configurazione Database PostgreSQL
spring.datasource.url=jdbc:postgresql://localhost:5432/easypricer
spring.datasource.username=easypricer
spring.datasource.password=easypricer
spring.datasource.driver-class-name=org.postgresql.Driver

# Configurazione JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.highlight_sql=true

# Configurazione Logging
logging.level.org.hibernate.SQL=INFO
logging.level.org.hibernate.orm.jdbc.bind=INFO
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=INFO
```

Usa il codice con cautela.



Note rapide sulla conversione:

AI Mode Tutti Immagini Video Notizie Altro ▾



Hai già aggiunto la **dipendenza del modulo** `easy_pricer_core` nel `pom.xml` progetto principale come avevamo visto prima?



Le risposte dell'AI potrebbero contenere errori. Scopri di più



come tradurre in inglese EASY PRICER SRV AVVIATO
CORRETTAMENTE

La traduzione più naturale e professionale in ambito informatico è:

EASY PRICER SRV STARTED SUCCESSFULLY

Se vuoi renderlo ancora più "standard" per un log di Spring Boot, puoi usare:

- **EASY PRICER SRV IS UP AND RUNNING** (più colloquiale: "è attivo e funzionante")
- **EASY PRICER SRV STARTED ON PORT 8080** (molto comune per i server web)

Piccola nota: Nel codice che abbiamo scritto prima, potresti aggiornare il

`System.out.println` così:

java

```
Svstem.out.println("=====");
```


AI Mode Tutti Immagini Video Notizie Altro ▾



Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



ho questo output . _____
^\\ / ____'_____()____ _ \\ \\ \\ ...



Ottimo! L'applicazione è partita correttamente: il server **Tomcat** è attivo sulla porta **8080** e hai anche un server **gRPC** attivo sulla **9090** (ereditato dal Core).

Tuttavia, l'ultimo errore nel log indica un problema di comunicazione con uno dei tuoi endpoint:

L'errore: `MethodArgumentTypeMismatchException`

Hai provato a chiamare un URL passando la stringa `"NASD_30_360"` , ma il tuo Controller si aspettava un numero intero (`Integer`).

Probabilmente hai chiamato:

```
http://localhost:8080/asset_class/r1/NASD_30_360
```

Ma il tuo metodo nel controller è definito così:

java

```
@GetMapping("/asset_class/r1/{id}")  
public ResponseEntity findByIdAssetClass(@PathVariable("id") Integer idA
```

AI Mode Tutti Immagini Video Notizie Altro ▾



java

```
@GetMapping("/asset_class/code/{code}")
public ResponseEntity findByCode(@PathVariable("code") String code) {
    // Dovrai implementare questo metodo nel DAO
}
```

Usa il codice con cautela.



Note sui Warning nel log:

1. **PostgreSQL Version:** Hibernate ti avvisa che la versione 9.6 di Postgres è vecchia. Se puoi, aggiorna il database a Postgres 12 o superiore, altrimenti ignora pure il warning.
2. **Open-in-View:** Per rimuovere il warning giallo su `spring.jpa.open-in-view`, aggiungi questa riga al tuo `application.properties` :
`spring.jpa.open-in-view=false`

Vuoi che ti aiuti a modificare il **Controller** per gestire la ricerca tramite quel codice stringa?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



nel prgetto precedente avevo questa configurazione <bean

AI Mode Tutti Immagini Video Notizie Altro ▾



java

```
package ph.alephzero.finance.products.forward;

import org.springframework.stereotype.Component;

@Component
public class BondForward {
    // ...
}
```

Usa il codice con cautela.



2. La strada "di configurazione" (Se non puoi modificare le classi)

Se quelle classi fanno parte di una libreria esterna (JAR) e non puoi aggiungere annotazioni, devi creare una classe di configurazione nel tuo progetto principale. Crea una classe chiamata ad esempio `ProjectConfig.java` :

java

```
package org.softcaster.easy_pricer_srv.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import ph.alephzero.finance.products.forward.BondForward;
import ph.alephzero.finance.products.forward.ForexForward;
```

AI Mode Tutti Immagini Video Notizie Altro ▾



```
        return new ForexForward();  
    }  
}
```

Usa il codice con cautela.



Quale scegliere?

- Usa la **n. 1** se le classi sono "tue" e vuoi che Spring le gestisca in modo trasparente.
- Usa la **n. 2** se vuoi mantenere un punto centrale dove vedere tutti i bean dichiarati, simile a quello che facevi con l'XML.

In entrambi i casi, potrai poi usarle nei tuoi controller o servizi usando l'abituale `@Autowired`.

Hai altre classi o **configurazioni particolari** (come DataSource o Transaction Manager) nel vecchio XML che dobbiamo spostare?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



il bean era annotato come `@Autowired`
`@Qualifier("bondForward")`

AI Mode Tutti Immagini Video Notizie Altro ▾



java

```
@Configuration
public class ProjectConfig {

    @Bean(name = "bondForward") // Il nome qui deve corrispondere al @Qu
    public BondForward bondForward() {
        return new BondForward();
    }

    @Bean(name = "forexForward")
    public ForexForward forexForward() {
        return new ForexForward();
    }
}
```

Usa il codice con cautela.



Come usarlo nel tuo codice

Nel punto in cui devi utilizzare questi oggetti (ad esempio in un Service o in un Controller), l'annotazione rimarrà identica a prima:

java

```
@Autowired
@Qualifier("bondForward")
private BondForward bondForward;
```

Usa il codice con cautela.



AI Mode Tutti Immagini Video Notizie Altro ▾



parametri nel costruttore (o altri bean), puoi passarli semplicemente come argomenti del metodo annotato con `@Bean`.

Hai altri bean con **scope particolari** (come `prototype` o `session`) nel vecchio XML?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



ho questo errore Field calculator in
org.softcaster.easy_pricer_srv.calc.BondForwardCalculator ... ▾

L'errore accade perché Spring non sta "scansionando" il package `ph.alephzero` oppure non ha trovato una classe di configurazione che dichiari esplicitamente quel bean.

Visto che la classe si trova in un package diverso (`ph.alephzero`), hai due modi per risolvere:

Soluzione A: Creare la classe di configurazione (Consigliata)

Questa è la soluzione più sicura se `ph.alephzero` fa parte di un JAR esterno. Crea questa classe nel tuo progetto principale:

AI Mode Tutti Immagini Video Notizie Altro ▾



```
@Bean(name = "bondForward")
public BondForward bondForward() {
    return new BondForward();
}
}
```

Usa il codice con cautela.



Soluzione B: Estendere lo Scan dei componenti

Se puoi modificare la classe `BondForward` aggiungendo `@Component` , devi anche dire a Spring Boot di guardare dentro quel package nella tua classe Main:

java

```
// Nella tua classe Easy_pricer_srv
@SpringBootApplication
@ComponentScan(basePackages = {"org.softcaster", "ph.alephzero"}) // Agg
public class Easy_pricer_srv { ... }
```



Usa il codice con cautela.



Perché la Soluzione A è meglio?

Nel tuo vecchio progetto XML usavi `<bean id="bondForward" ...>` , che equivale esattamente alla **Soluzione A**. Usando `@Configuration` , mantieni la logica separata e non hai bisogno di aggiungere annotazioni Spring a classi che magari appartengono a librerie finanziarie esterne.

AI Mode Tutti Immagini Video Notizie Altro ▾



Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



io avevo fatto questa classe package
org.softcaster.easy_pricer;...



Il problema non è necessariamente il nome del package "config", ma il fatto che Spring deve riuscire a **trovare** questa classe durante la scansione iniziale.

Affinché quella classe funzioni, devi controllare due cose:

1. La posizione del Package

Nel tuo file `Easy_pricer_srv.java` (la classe Main), abbiamo impostato:

```
@ComponentScan(basePackages = "org.softcaster") .
```

- Se la tua classe `ProjectConfig` si trova in package `org.softcaster.easy_pricer;`, Spring **dovrebbe** trovarla perché inizia con `org.softcaster` .
- Tuttavia, per convenzione e ordine, ti consiglio di spostarla in:
`org.softcaster.easy_pricer_srv.config`
(visto che il tuo progetto principale si chiama `easy_pricer_srv`).

2. Perché non la vede? (Il motivo più probabile)

Se la classe è nel package corretto ma l'errore persiste, controlla se hai fatto un

AI Mode Tutti Immagini Video Notizie Altro ▾



3. Verifica che non ci siano altre classi `ProjectConfig` duplicate in giro.

Un test rapido

Nella tua classe Main, prova ad aggiungere questo log nel metodo `main` per vedere se il bean esiste davvero:

java

```
var context = SpringApplication.run(Easy_pricer_srv.class, args);  
System.out.println("Contiene bondForward? " + context.containsBean("bond
```



Usa il codice con cautela.



Se stampa `false` , significa che la classe `ProjectConfig` è stata ignorata dallo scanner. In quel caso, il problema è quasi certamente la gerarchia dei package rispetto a dove si trova la tua classe `@SpringBootApplication` .

La tua classe Main si trova nel package `org.softcaster.easy_pricer_srv` ? Se sì, e la config è in `org.softcaster.easy_pricer` , assicurati che lo scanner parta dal livello più alto (`org.softcaster`).

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



I prossimi passi consigliati.

1. **Test degli endpoint:** Prova a chiamare uno dei tuoi URL REST (ad esempio quello delle `asset_class`) tramite browser o un tool come Postman per verificare che la connessione al database PostgreSQL funzioni a dovere.
2. **Monitoraggio Console:** Tieni d'occhio i log. Se hai attivato `show-sql=true` , vedrai scorrere le query ogni volta che interroghi i DAO.
3. **Gestione Errori:** Se dovessi ricevere ancora errori di "Type Mismatch" (come quello della stringa al posto dell'Integer), ricordati di controllare le definizioni dei `@PathVariable` nei tuoi Controller.

Hai qualche altro **pezzo del vecchio XML** (magari filtri, interceptor o task schedulati) che dobbiamo convertire, o il grosso della migrazione è completato? **bolding su interceptor o task.**

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)

